Q1. Load the given retail sales dataset into a Pandas DataFrame and display the first 5 rows along with the dataset information to understand its structure.

In [2]:
```
import pandas as pd
```

In [4]:
```
df = pd.read_csv("retail_dataset .csv")
```

In [5]:
```
print(df.head())
```

```
  Transaction ID Customer ID Product Category  Quantity    Price        Date  \
0           T001        C015            Books       7.0   167.0  02-02-2023
1           T002        C004             Toys       4.0   374.0  17-02-2023
2           T003        C033        Furniture       8.0  1059.0  17-05-2024
3           T004        C047        Groceries       5.0   304.0  11-02-2024
4           T005        C022            Books       7.0   282.0  27-07-2024

  Region
0  North
1  South
2  South
3  South
4   West
```

In [6]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    100 non-null    object
 1   Customer ID       100 non-null    object
 2   Product Category  97 non-null     object
 3   Quantity          98 non-null     float64
 4   Price             95 non-null     float64
 5   Date              100 non-null    object
 6   Region            98 non-null     object
dtypes: float64(2), object(5)
memory usage: 5.6+ KB
None
```

Q2. Inspect the dataset and identify missing values in each column. Display the total number of missing values present in the dataset.

In [7]:
```
missing_values = df.isnull().sum()
print(missing_values)
```

```
Transaction ID      0
Customer ID         0
Product Category    3
Quantity            2
Price               5
Date                0
Region              2
dtype: int64
```

In [8]:
```
total_missing = df.isnull().sum().sum()
print("Total Missing Values:", total_missing)
```

```
Total Missing Values: 12
```

Q3. Replace missing values in the Quantity and Price columns with their respective mean values using Pandas.

In [11]:
```
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].mean())
df['Price'] = df['Price'].fillna(df['Price'].mean())
```

In [12]:
```
print(df[['Quantity', 'Price']].isnull().sum())
```

```
Quantity    0
Price       0
dtype: int64
```

Q4. Remove all rows where the Product Category or Region column contains missing values.

In [13]:
```
df = df.dropna(subset=['Product Category', 'Region'])
```

In [14]:
```
print(df[['Product Category', 'Region']].isnull().sum())
```

```
Product Category    0
Region              0
dtype: int64
```

Q5. Create a new column called Revenue using NumPy, where: Revenue = Quantity × Price Display the updated dataset with the new column.

In [15]:
```
import numpy as np
```

In [16]:
```
df['Revenue'] = np.multiply(df['Quantity'], df['Price'])
```

In [17]:
```
print(df.head())
```

```
  Transaction ID Customer ID Product Category  Quantity    Price        Date  \
0           T001        C015             Books       7.0   167.0  02-02-2023
1           T002        C004              Toys       4.0   374.0  17-02-2023
2           T003        C033         Furniture       8.0  1059.0  17-05-2024
3           T004        C047         Groceries       5.0   304.0  11-02-2024
4           T005        C022             Books       7.0   282.0  27-07-2024

  Region  Revenue
0  North   1169.0
1  South   1496.0
2  South   8472.0
3  South   1520.0
4   West   1974.0
```

Q6. Using NumPy, calculate the total revenue generated from all transactions in the dataset.

In [18]:
```
total_revenue = np.sum(df['Revenue'])
```

In [19]:
```
print("Total Revenue:", total_revenue)
```

```
Total Revenue: 465529.66058002145
```

Q7. Group the dataset by Product Category and calculate the total revenue generated by each category.

In [20]:
```
category_revenue = df.groupby('Product Category')['Revenue'].sum()

print(category_revenue)
```

```
Product Category
Beauty          27647.000000
Books           29597.938776
Clothing        85006.000000
Electronics     93204.315789
Furniture      129664.526316
Groceries       11978.000000
Sports          34484.000000
Toys            53947.879699
Name: Revenue, dtype: float64
```

Q8. Based on the revenue calculated for each product category, identify the top 3 categories with the highest revenue and the bottom 3 categories with the lowest revenue.

In [21]:
```
category_revenue = df.groupby('Product Category')['Revenue'].sum()
```

In [22]:
```
top_3 = category_revenue.sort_values(ascending=False).head(3)
```

In [23]:
```
bottom_3 = category_revenue.sort_values(ascending=True).head(3)
```

In [24]:
```
print("Top 3 Categories by Revenue:")
print(top_3)
```

```
Top 3 Categories by Revenue:
Product Category
Furniture      129664.526316
Electronics     93204.315789
Clothing        85006.000000
Name: Revenue, dtype: float64
```

In [25]:
```
print("\nBottom 3 Categories by Revenue:")
print(bottom_3)
```

```
Bottom 3 Categories by Revenue:
Product Category
Groceries    11978.000000
Beauty       27647.000000
Books        29597.938776
Name: Revenue, dtype: float64
```

Q9. Group the dataset by Region and calculate the total revenue generated in each region. Identify the region with the highest revenue and the region with the lowest revenue.

In [26]:
```
region_revenue = df.groupby('Region')['Revenue'].sum()
```

In [27]:
```
print("Revenue by Region:")
print(region_revenue)
```

```
Revenue by Region:
Region
East     126232.044039
North    153244.353383
South    104855.736842
West      81197.526316
Name: Revenue, dtype: float64
```

In [28]:
```
highest_region = region_revenue.idxmax()
highest_revenue = region_revenue.max()
```

In [29]:
```
lowest_region = region_revenue.idxmin()
lowest_revenue = region_revenue.min()
```

In [30]:
```
print("\nRegion with Highest Revenue:")
print(f"{highest_region}: {highest_revenue}")
```

```
Region with Highest Revenue:
North: 153244.35338345866
```

In [31]:
```
print("\nRegion with Lowest Revenue:")
print(f"{lowest_region}: {lowest_revenue}")
```

```
Region with Lowest Revenue:
West: 81197.52631578947
```

Q10. Extract the month from the Date column using Pandas and group the dataset by month to calculate the total monthly revenue. Also, calculate the mean, median, and standard deviation of the Revenue column using NumPy.

In [32]:
```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
```

In [33]:
```
df['Month'] = df['Date'].dt.month_name()
```

In [34]:
```
monthly_revenue = df.groupby('Month')['Revenue'].sum()
```

In [35]:
```
print("Monthly Revenue:")
print(monthly_revenue)
```

```
Monthly Revenue:
Month
April        57298.526316
August       76958.000000
December     43339.000000
February     33028.000000
January      43994.000000
July         47851.292159
June         28190.000000
March        10557.000000
May          27915.000000
November     27662.736842
October      38017.000000
September    30719.105263
Name: Revenue, dtype: float64
```

In [36]:
```
mean_revenue = np.mean(df['Revenue'])
median_revenue = np.median(df['Revenue'])
std_revenue = np.std(df['Revenue'])
print("Mean Revenue:", mean_revenue)
print("Median Revenue:", median_revenue)
print("Standard Deviation of Revenue:", std_revenue)
```

```
Mean Revenue: 4900.312216631804
Median Revenue: 3282.1052631578946
Standard Deviation of Revenue: 4721.171795257337
```

In [ ]:
```

```